

Exercício de Fixação — Subprogramas

Página / Slide 21

Aluno: Rodrigo Del Padre**RA:** 24022092-2**Curso:** Engenharia de Software (6º Semestre)**Turma:** B**Disciplina:** Paradigmas de Programação**Referência:** R. Sebesta, 11ª ed., Caps. 9 e 10

Trecho 1 — Python

Código analisado:

```
def adicionar(item, lista=[]):  
    lista.append(item)  
    return lista  
  
print(adicionar(1))  
print(adicionar(2))
```

(a) Qual é a saída?

```
[1]  
[1, 2]
```

(b) Qual conceito da aula explica?

Conceito Central: *Binding Time* de Parâmetros Padrão e Mutabilidade de Objetos (Sebesta, 11ª ed., §9.2.3, p. 367–371; Slide 5).

- **Avaliação em Tempo de Definição:** Em Python, expressões atribuídas como valores padrão de parâmetros (`lista=[]`) são avaliadas uma única vez durante a definição da função (tempo de carga do módulo), e não a cada chamada.
- **Instância Compartilhada no Heap:** O interpretador aloca um único objeto `lista` no *heap*. Nas invocações sem o segundo argumento, o parâmetro formal `lista` é vinculado a essa mesma referência.
- **Efeito Colateral Acumulativo:** Na 1ª chamada, `.append(1)` adiciona 1 à lista. Na 2ª chamada, `.append(2)` atua sobre o mesmo objeto persistido, resultando em `[1, 2]`.
- **Boa Prática em Engenharia de Software:** Para evitar o compartilhamento indesejado de estado, adota-se o padrão sentinela: `def adicionar(item, lista=None): if lista is None: lista = []`.

Trecho 2 — Java

Código analisado:

```
static void zera(int[] v, int n) {  
    v[0] = 0;  
    n = 0;  
}  
  
int[] v = {5, 5};
```

```
int n = 5;
zera(v, n);
System.out.println(v[0] + " " + n);
```

(a) Qual é a saída?

0 5

(b) Qual conceito da aula explica?

Conceito Central: Passagem Estrita por Valor (*Pass-by-Value*) de Tipos Primitivos vs. Referências a Objetos (Sebesta, 11ª ed., §9.5.4, p. 383; Slide 11).

- **Primitivo (int n):** O valor escalar 5 é copiado na pilha para o registro de ativação do método zera. A reatribuição `n = 0;` altera unicamente essa cópia local. Ao término do método, a pilha é liberada e a variável `n` do chamador permanece inalterada com 5.
- **Array/Objeto (int[] v):** Em Java, arrays são objetos no *heap*. O que é passado por valor é o valor da **referência** (endereço de memória). Como o parâmetro formal `v` e o chamador compartilham a mesma referência, a alteração `v[0] = 0;` muda a memória no *heap* diretamente.
- **Resultado Final:** `v[0]` reflete a modificação e passa a valer 0, enquanto o primitivo `n` permanece 5.